

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1105

COURSE OUTCOME COVERED

C02: Design UML diagrams to represent system requirements and architecture. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS ANSWERED: 4 of 6 (Use Case, Class, Activity, State Diagrams) Total Marks: 40

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1: Use Case Diagram — Smart Banking System (10 Marks)

A Smart Banking System allows customers to create accounts, transfer funds, pay bills, and view transaction history. Bank employees verify customer details, while administrators manage user accounts and banking services. The system requirements are only partially documented.

a) Identify all actors involved in the system.

An actor is any external entity — a human role or another system — that interacts with the system to achieve a goal but lies outside the system boundary. Four actors are identified for the Smart Banking System:

Actor	Role in the System
Customer	Primary actor who owns an account and initiates account creation, fund transfers, bill payments, and history viewing.
Bank Employee	Internal staff actor who verifies customer identity and documents during account-related processes.
Administrator	A specialised Bank Employee with elevated privileges who manages user accounts and configures banking services. Modelled as a generalisation of Bank Employee since it inherits the employee's login/verification behaviour and adds administrative capability.
Payment Gateway (external system)	A secondary/supporting actor — an external financial system invoked whenever a monetary transaction (bill payment or fund transfer) must be settled outside the bank's own ledger.

b) List the primary use cases.

- Create Account — customer opens a new savings/current account.
- Login / Authenticate User — common entry point used by every actor before performing a secured operation.
- Transfer Funds — move money between the customer's own accounts or to a third party.
- Pay Bills — settle utility/merchant bills linked to the account.
- View Transaction History — display past debit/credit entries.
- Generate Account Statement — produce a downloadable/printable summary of transactions.
- Verify Customer Details — employee cross-checks KYC documents during account opening.
- Process Payment via Gateway — hands off a transaction to the external payment gateway for settlement.
- Manage User Accounts — administrator creates, suspends, or deletes customer/employee logins.
- Manage Banking Services — administrator configures products such as interest rates or service charges.

c) Define relationships (include, extend, generalization).

Relationship	Justification
Transfer Funds --include--> Authenticate User	Every fund transfer mandatorily triggers authentication; the base use case cannot complete without it.
Pay Bills --include--> Authenticate User	Bill payment always requires the user to be logged in and verified first.
View Transaction History --include--> Authenticate User	History can only be viewed after successful login.
Create Account --include--> Verify Customer Details	Account creation is incomplete unless the employee verification sub-flow runs, so it is included, not optional.
Generate Account Statement --extend--> View Transaction History	Generating a statement is an optional add-on that a customer may or may not invoke while viewing history; it extends the base flow only under the condition that the customer clicks 'Export'.
Process Payment via Gateway --extend--> Pay Bills / Transfer Funds	The external gateway call is triggered only for transactions that need real-time third-party settlement, so it optionally extends both base use cases.
Administrator --generalization--> Bank Employee	Administrator is a specialised kind of Bank Employee: it inherits Login and Verify Customer Details and adds Manage User Accounts and Manage Banking Services.

d) Construct a complete Use Case Diagram representing system functionality.

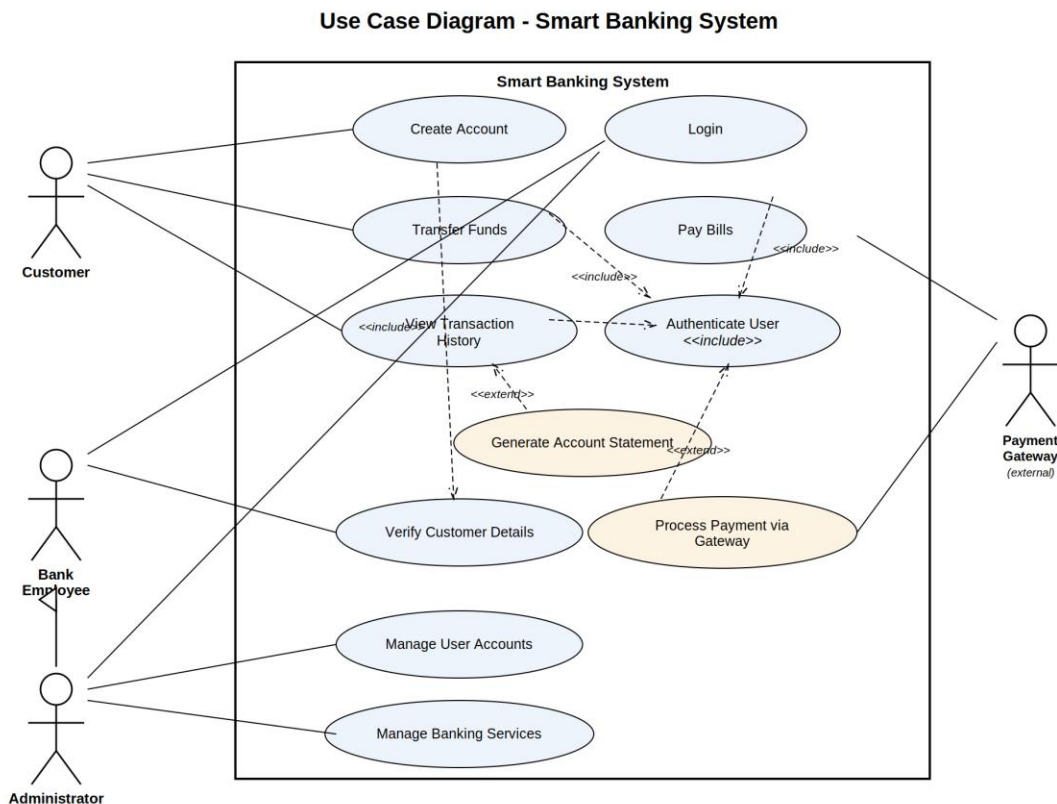


Figure 1.1 — Use Case Diagram for the Smart Banking System

The diagram places the system boundary around all ten use cases. Customer associates with Create Account, Transfer Funds, and View Transaction History; Bank Employee associates with Verify Customer Details and inherits into Administrator, who additionally associates with Manage User Accounts and Manage Banking Services. The dashed include arrows point from the base use case toward the mandatory sub-behaviour (Authenticate User, Verify Customer Details), while the dashed extend arrows point from the optional use case back toward the base use case they augment (Generate Account Statement, Process Payment via Gateway). The Payment Gateway actor sits outside the boundary and is reached only through the extend path, reflecting that it is invoked conditionally rather than on every transaction.

Question 2: Class Diagram — Online Vehicle Rental System (10 Marks)

An Online Vehicle Rental System consists of entities such as *Vehicle*, *Customer*, *Reservation*, *Payment*, and *RentalAgent*. The relationships among these entities have not been clearly defined.

a) Identify classes with appropriate attributes and methods.

Class	Attributes / Methods
Vehicle (abstract)	vehicleId, regNumber, rentalRate, status checkAvailability(), updateStatus()
Car	seatingCapacity, fuelType calculateFare() — inherits Vehicle
Bike	cc, helmetIncluded calculateFare() — inherits Vehicle
Customer	customerId, name, licenseNo, contact register(), makeReservation(), viewHistory()
Reservation	reservationId, startDate, endDate, status createReservation(), cancelReservation(), confirmBooking()
Payment	paymentId, amount, mode, paymentDate processPayment(), generateReceipt()
RentalAgent	agentId, name approveReservation(), assignVehicle(), handleReturn()
RentalAgency	agencyId manageFleet()

Vehicle is modelled as an abstract superclass because the system never instantiates a generic 'Vehicle' — every rentable unit is concretely a Car or a Bike, each with its own fare-calculation logic (fuel-based vs. cc-based pricing), which is a textbook case for generalisation/inheritance.

b) Define relationships (association, aggregation, inheritance, composition where applicable).

Relationship	Justification
Car / Bike --inheritance--> Vehicle	Car and Bike are specialised kinds of Vehicle and inherit its common attributes and the checkAvailability() behaviour.
Customer --association--> Reservation	A customer makes reservations; the link is a simple association since both classes can exist independently of each other.
Reservation --association--> Vehicle	A reservation books exactly one vehicle for a date range; the vehicle is not owned by the reservation.
Reservation --composition--> Payment	A Payment record is meaningless and cannot exist without its owning Reservation — if the reservation is deleted, its payment record is deleted with it. This is a strong whole-part (composition) relationship, shown with a filled diamond.
RentalAgent --association--> Reservation	An agent processes/approves multiple reservations over time; neither class owns the other's lifecycle.
RentalAgency --aggregation--> Vehicle	The agency's fleet is made up of vehicles, but a vehicle could conceptually be transferred to another branch/agency and still exist — a whole-part relationship without strict lifecycle ownership, shown with a hollow diamond.

c) Specify multiplicity between classes.

Class Pair	Multiplicity
Customer — Reservation	1 to 0..* (a customer may have zero or many reservations; each reservation belongs to exactly one customer)
Reservation — Vehicle	1 to 1 (each reservation books exactly one vehicle for its duration)
Reservation — Payment	1 to 1 (each reservation produces exactly one payment record)
RentalAgent — Reservation	1 to 0..* (one agent can process many reservations)
RentalAgency — Vehicle	1 to 1..* (an agency owns at least one vehicle in its fleet)

d) Design a detailed Class Diagram for the system.

Class Diagram - Online Vehicle Rental System

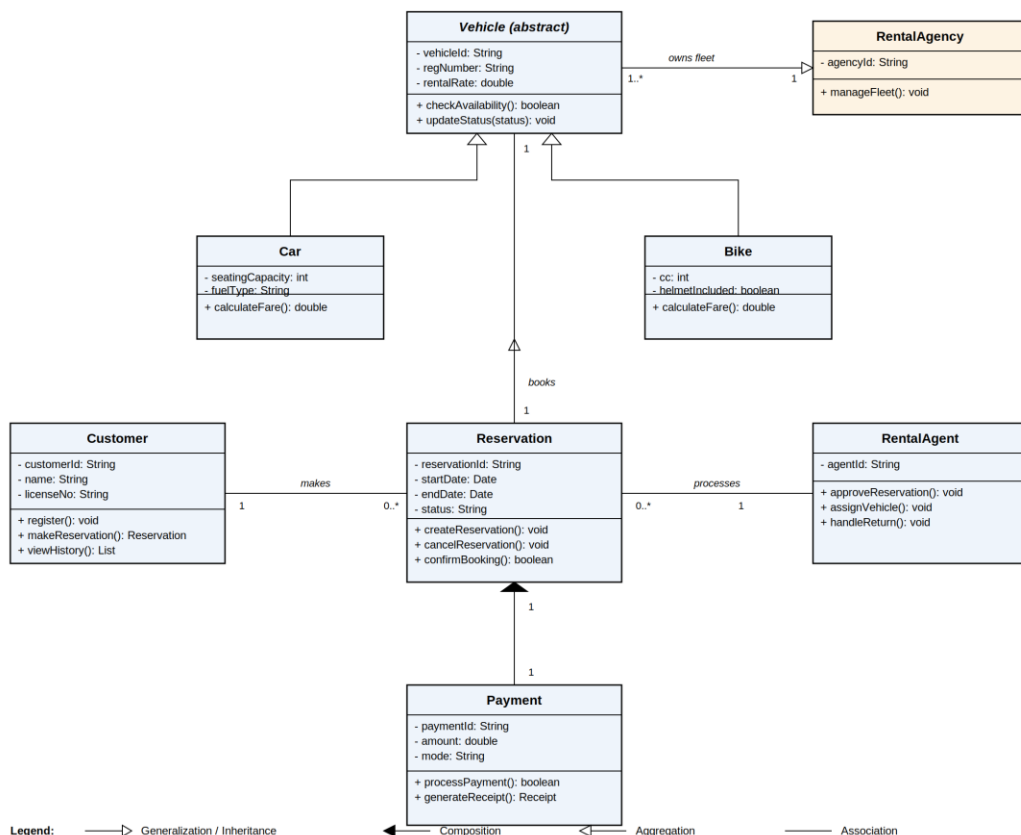


Figure 2.1 — Class Diagram for the Online Vehicle Rental System

The abstract Vehicle class sits at the top with Car and Bike inheriting below it via hollow-triangle generalisation arrows. Customer, Reservation, RentalAgent, and Payment form the operational core: Customer associates with Reservation (1 to 0..*), Reservation associates with Vehicle (1 to 1), and Reservation is composed of exactly one Payment (filled diamond, 1 to 1). RentalAgent associates with Reservation (1 to 0..*) to represent agent-side processing, and RentalAgency aggregates the Vehicle fleet (hollow diamond, 1 to 1..*), reflecting that vehicles are fleet assets that could outlive a single agency assignment.

Question 3: Activity Diagram — Job Recruitment Portal (10 Marks)

A Job Recruitment Portal enables applicants to register, upload resumes, apply for jobs, attend interviews, and receive offer letters. Recruiters verify applications before scheduling interviews. The business workflow needs optimization.

a)) Identify all activities in the recruitment workflow.

- Register Account (Applicant)
- Upload Resume (Applicant)
- Apply for Job (Applicant)
- Verify Application (Recruiter)
- Send Rejection Notification (System, if invalid application)
- Schedule Interview (Recruiter)
- Attend Technical Interview and Attend HR Interview (Applicant — run in parallel)
- Evaluate Candidate (Recruiter)
- Send Offer Letter / Send Rejection Letter (System)
- Confirm Joining or Decline Offer (Applicant)

b)) Add decision nodes for selection and rejection.

Three decision (diamond) nodes optimise the workflow by short-circuiting invalid or unsuccessful applicants early instead of carrying them through every downstream step:

- **Decision 1 — Valid?** placed right after Verify Application. An [Invalid] application is rejected immediately, saving recruiter time that would otherwise be spent scheduling interviews for an ineligible candidate.
- **Decision 2 — Selected?** placed after Evaluate Candidate. A [No] branch sends a rejection letter; a [Yes] branch proceeds to the offer stage.
- **Decision 3 — Accept?** placed after the applicant receives the offer, branching to Confirm Joining or Decline Offer.

c)) Include parallel activities wherever applicable.

Once an interview is scheduled, the Technical Interview and the HR Interview do not depend on each other and can be conducted independently (often by different panels on the same day), so a fork bar splits the flow into two concurrent activities, and a join bar synchronises them before the recruiter moves on to Evaluate Candidate — this is the key optimisation requested, since running the interviews in parallel rather than sequentially reduces the overall time-to-decision for the applicant.

d)) Design an optimized Activity Diagram representing the complete workflow.

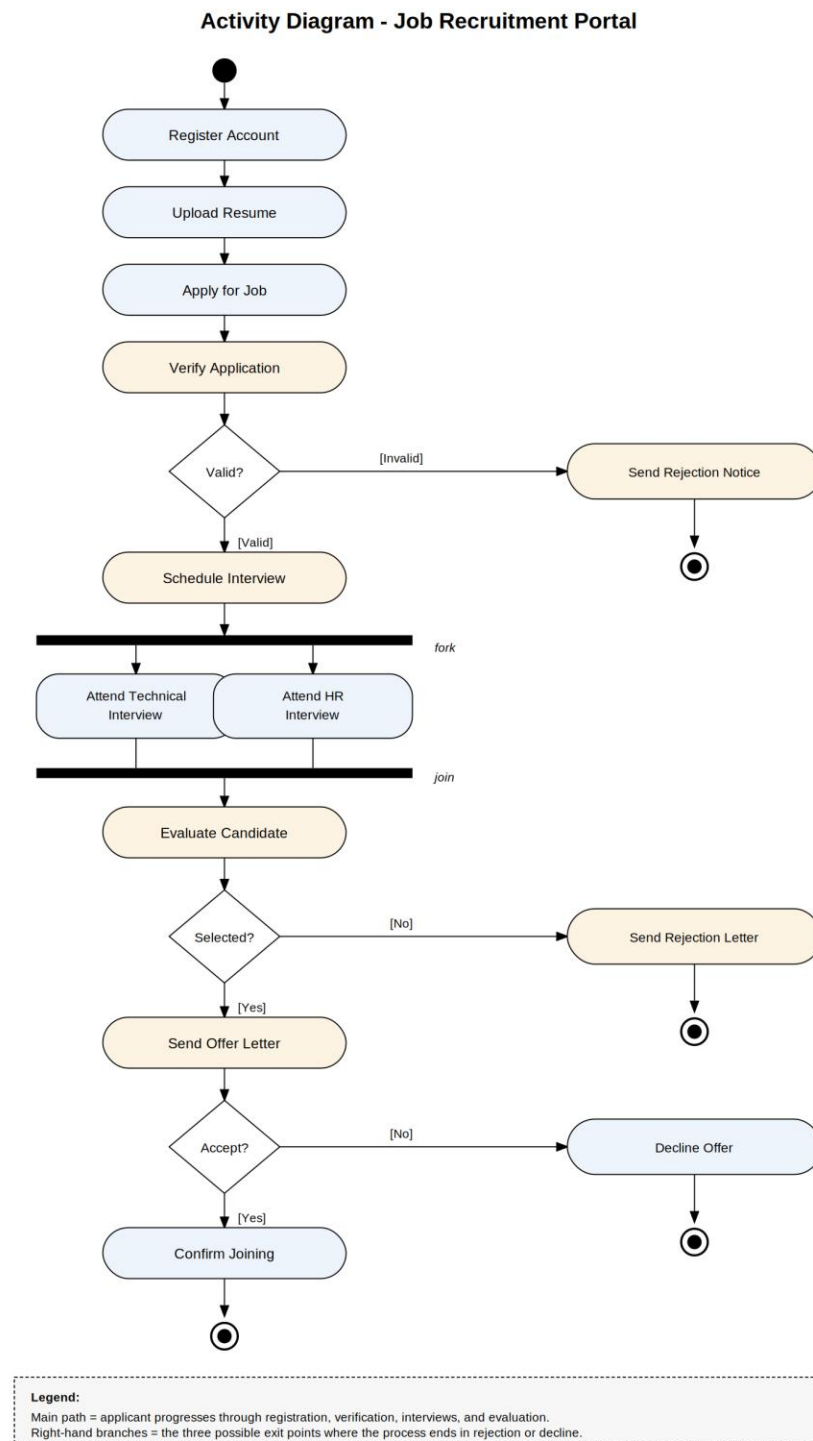


Figure 3.1 — Activity Diagram for the Job Recruitment Portal

The diagram is drawn as a single main flow so the overall process stays easy to follow: it starts at the initial node and runs straight down through Register Account, Upload Resume, Apply for Job, and Verify Application. At each decision diamond (Valid?, Selected?, Accept?) the 'happy path' continues straight down the centre, while the rejection/decline outcome branches off to the right into its own short box and ends at its own activity-final node — so the three possible endings never cross back through the main line. The fork and join bars in the centre split the flow into Attend Technical Interview and Attend HR Interview running in parallel, then resynchronise before Evaluate Candidate, which is the optimisation requested by the question.

Question 4: State Diagram — Smart Parking Management System (10 Marks)

A Smart Parking Management System operates through states such as *Parking Slot Available*, *Vehicle Entered*, *Parking Allocated*, *Payment Pending*, *Payment Completed*, and *Vehicle Exited*. The system behavior during parking operations needs to be modeled.

a) *Identify all possible states of the system.*

State	Description
Slot Available	The default/idle state — at least one parking bay is free and the system is ready to admit a vehicle.
Vehicle Entered	A vehicle has been sensed at the entry gate but has not yet been assigned a specific bay.
Parking Allocated	A specific slot number has been assigned and displayed to the driver; the vehicle is parked.
Payment Pending	The driver has requested to exit and the system is calculating/awaiting the parking fee.
Payment Completed	The fee has been successfully collected (cash, card, or app-based).
Vehicle Exited	The barrier has opened and the vehicle has left, freeing the slot.

b) *Define events triggering state transitions.*

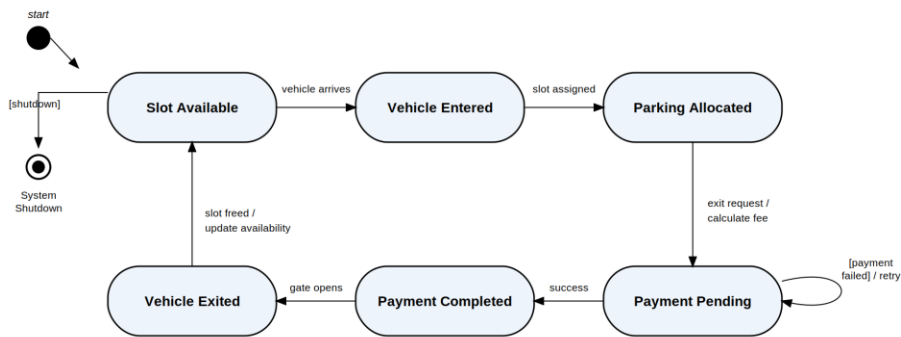
Transition	Triggering Event / Guard
Slot Available -> Vehicle Entered	Event: vehicle arrives / entry sensor detects vehicle
Vehicle Entered -> Parking Allocated	Event: system assigns a free slot and displays the slot number
Parking Allocated -> Payment Pending	Event: driver initiates exit / scans exit request
Payment Pending -> Payment Completed	Event: payment transaction succeeds
Payment Pending -> Payment Pending (self-transition)	Event: payment fails, prompting a retry without changing state
Payment Completed -> Vehicle Exited	Event: boom barrier opens and vehicle physically leaves
Vehicle Exited -> Slot Available	Event: slot sensor confirms the bay is empty and availability is updated, re-arming the cycle for the next vehicle

c) *Include initial and final states.*

A solid black circle marks the initial pseudostate, transitioning into Slot Available when the system first comes online. Because parking is a repeating cycle, Slot Available is deliberately modelled as both the initial condition and the state re-entered after every Vehicle Exited transition. A final state (circled dot) is included to represent system shutdown/session end, reachable from Vehicle Exited — this closes the diagram formally per UML notation without implying that the physical car park itself stops operating after one vehicle.

d) *Construct a State Transition Diagram representing system behavior.*

State Transition Diagram - Smart Parking Management System



Note:

Slot Available is the initial condition of the system, the state re-entered after every vehicle exits, and the only state from which the system can transition to the final state (on shutdown). The Payment Pending state has a self-transition guarded by [payment failed], allowing the customer to retry before payment succeeds.

Figure 4.1 — State Transition Diagram for the Smart Parking Management System

The six operational states are arranged in a clockwise cycle from Slot Available through Vehicle Entered, Parking Allocated, Payment Pending, Payment Completed, and Vehicle Exited, before looping back to Slot Available. Payment Pending carries a self-transition guarded by [payment failed] to model retries without corrupting the flow. The diagram opens at the initial pseudostate, which feeds into Slot Available, and Slot Available is also the single point from which the system can drop into the final state under a [shutdown] guard — kept as a separate short branch on the left so it never crosses the main vehicle-cycle transitions.